Project – High Level Design
On
Multi Agent Educational System

Course Name: Agentic AI

Institution Name: **Medicaps University Datagami Skill Based Course**

Sr no	Student Name	Enrolment Number
1	NAMAN SHARMA	EN22IT301058
2	JAYENDRA VERMA	EN22IT301043
3	HITENDRA SAMEDIYA	EN24CA5030067
4	HANSRAJ SINGH RAJPUT	EN24CA5030055
5	ASHISH NAGDA	EN24CA5030026
6	ATUL MANDLOI	EN24CA5030031

Project Number: AAI- 04

Insdustry Mentor Name : Mr. Ashwin

University Mentor Name: Prof. Ajeet Sir

Academic Year:2025-26

Group Name: G13D12

Abstract

The **Multi-Agent Education System** is an AI-driven platform designed to enhance the learning experience by generating structured and context-aware educational content using a collaborative multi-agent approach. Traditional learning systems often rely on static resources, which lack personalization and real-time interaction, making it difficult for learners to fully understand complex topics. This system addresses these limitations by leveraging the concept of Agentic AI, where multiple intelligent agents work together to process user queries and produce meaningful responses.

The system follows a layered architecture consisting of a user interface, backend processing unit, agent orchestration module, and an AI-powered language model layer. When a user submits a query, it is processed through an orchestrated workflow in which the Researcher Agent gathers relevant information and the Writer Agent transforms it into structured and easy-to-understand content. The integration of a Large Language Model (LLM) further enhances the quality of responses by improving reasoning, language clarity, and contextual accuracy.

Additionally, the system incorporates memory and state management to maintain consistency across interactions, ensuring coherent and relevant outputs. The modular design of the system makes it scalable, maintainable, and adaptable for future enhancements, such as the addition of more specialized agents or advanced knowledge sources.

Overall, the proposed system provides an interactive, efficient, and intelligent solution for personalized learning by delivering high-quality educational content, thereby improving user engagement and knowledge comprehension.

Table of Contents

S. No.	Topic	Page
Chapter 1	Introduction	ii
1	Scope of the document	iii
2	Intented Audience	iv
3	System Overview	v
Chapter 2	System Design	vi
1	Application Design	vii
2	Process Flow	viii
3	Information Flow	ix
4	Component Design	x
5	Key Design Considerations	xi
6	API Catalogue	
Chapter 3	Data Design	1
1	Data Model	
2	Data Access Mechanism	
3	Data Retention Policies	
4	Data Migration	
Chapter 4	Interfaces	
Chapter 5	States and Session Management	
Chapter 6	Caching	
Chapter 7	Non – Functional Requirement	
1	Security Aspects	
2	Performance Aspects	
Chapter 8	Reference	

CHAPTER 1: INTRODUCTION

The **Multi-Agent Education System** is an AI-based application designed to provide structured and personalized educational content using multiple intelligent agents. Traditional learning systems rely on static resources, which often lack interactivity and adaptability. This system overcomes those limitations by leveraging **Agentic AI**, where specialized agents collaborate to process user queries and generate meaningful responses.

The system integrates a Large Language Model (LLM) to enhance reasoning and language generation. By combining agent collaboration with AI capabilities, the platform delivers accurate, clear, and easy-to-understand educational content, improving the overall learning experience.

Scope of the Document

This document presents the High-Level Design (HLD) of the Multi-Agent Education System, offering a comprehensive overview of the system's architecture, core components, workflows, and data handling mechanisms. It outlines how the system is structured and how different modules interact to process user queries and generate meaningful educational content.

The scope of this document includes the overall system architecture and design, a high-level description of individual components, the flow and processing of data across the system, the structure and purpose of APIs, and key non-functional requirements such as performance, scalability, and security. However, this document does not cover low-level implementation details such as specific code logic, algorithms, or technical configurations, as its primary focus is to provide a conceptual and architectural understanding of the system.

Intented Audience

This document is intended for a wide range of individuals involved in the design, development, evaluation, and understanding of the Multi-Agent Education System. It is particularly useful for system architects who need a clear view of the overall system structure and design approach. Developers can use this document to gain a high-level understanding of

the system before moving into implementation. Additionally, project evaluators and academic reviewers can refer to it to assess the architecture, workflow, and design decisions of the

system. It is also beneficial for stakeholders who want to understand the system's capabilities, functionality, and overall working without going into technical implementation details.

System Overview

The Multi-Agent Education System is designed as an intelligent multi-agent AI platform in which multiple specialized agents collaborate to generate structured and meaningful educational content. The system operates by accepting a user's query or topic as input and processing it through a coordinated workflow managed by an orchestration mechanism. Once the query is received, it is first analyzed and then forwarded to the Researcher Agent, which gathers relevant information from available knowledge sources.

This information is subsequently passed to the Writer Agent, which transforms the raw data into clear, well-structured, and easy-to-understand content. The processed content is further enhanced by a Large Language Model (LLM), which improves the quality, accuracy, and contextual relevance of the response. Finally, the system delivers the refined output to the user in an organized format. This collaborative multi-agent approach ensures higher accuracy, improved clarity, and context-aware responses, thereby enhancing the overall learning experience.

CHAPTER 2: SYSTEM DESIGN

2.1 Application Design

The system follows a layered architecture that ensures modularity, scalability, and ease of maintenance. It is divided into multiple layers, each responsible for a specific function within the system. The User Interface layer handles all user interactions, allowing users to input queries and view responses.

The Backend layer processes these requests and manages the overall workflow of the system. The Agent Orchestration layer controls the collaboration between different agents, ensuring that tasks are properly assigned and executed in sequence. Finally, the AI/LLM layer enhances reasoning capabilities and generates high-quality responses. This clear separation of responsibilities allows the system to be easily maintained, updated, and scaled in the future.

2.2 Process Flow

The process flow describes the step-by-step operation of the system from input to output. It begins when the user enters a query, which is received and processed by the backend. The request is then forwarded to the orchestrator, which assigns tasks to the appropriate agents.

The Researcher Agent retrieves relevant data based on the query, and this information is passed to the Writer Agent, which structures it into clear and understandable content. The generated content is further refined by the LLM to improve accuracy and language quality. Finally, the response is formatted properly and displayed to the user. This structured process ensures smooth execution and high-quality output generation.

2.3 Information Flow

The information flow in the system ensures efficient and seamless movement of data across all components. The process starts with the user input, which is sent to the backend for processing. From there, it moves to the orchestrator, which directs it to the Researcher Agent for data collection.

The gathered information is then passed to the Writer Agent for structuring and formatting. After that, the LLM enhances the response, and the formatted output is sent back to the user. This structured flow of information ensures consistency, accuracy, and efficient communication between all components of the system.

2.4 Component Design

The system consists of several key components, each responsible for a specific role in the overall workflow. The User Module handles user interaction and input/output operations. The Backend Server processes incoming requests and manages system operations. The

Orchestrator controls the workflow and coordinates communication between agents. The Researcher Agent is responsible for retrieving relevant data, while the Writer Agent converts that data into structured and readable content. The Memory Module maintains context and stores intermediate results to ensure consistency. The LLM Module enhances the response by improving reasoning and language quality. Finally, the Formatter prepares the output in a clear and structured format before presenting it to the user.

2.5 Key Design Considerations

Several important factors are considered while designing the system to ensure its effectiveness and reliability. Accuracy is maintained by using reliable data sources and ensuring correct information retrieval. Scalability is achieved through a modular architecture that supports multiple users and future expansion. Performance is optimized to provide fast response times and efficient processing.

Security measures are implemented to protect user data and ensure safe system operation. Maintainability is ensured through a clean and modular design, making it easy to update and debug the system. Consistency is maintained by following structured workflows, ensuring uniform output quality across different queries.

2.6 API Catalogue

The system provides a set of APIs to facilitate communication between different components and to handle user interactions. The Generate Response API allows users to submit queries and receive structured educational content as output. The Get Agent Info API provides information about the available agents within the system. The User History API enables

retrieval of past user interactions, helping in tracking previous queries and responses. The Store API is used to save user queries and generated responses for future reference. These APIs are designed to be simple, scalable, and easy to integrate with the system. The traditional finance learning into an intelligent and engaging AI-driven experience.

CHAPTER 3: DATA DESIGN

3.1 Data Model

The data model of the Multi-Agent Education System is designed to organize and manage information efficiently while supporting the system's workflow. It primarily includes entities such as user data, query data, response data, and agent data. User data stores basic information such as user ID and interaction history.

Query data contains the input provided by the user along with timestamps. Response data stores the generated educational content produced by the system. Agent data maintains details about different agents, including their roles and responsibilities. These entities are logically connected to ensure smooth data flow and consistency across the system, enabling efficient retrieval and processing of information.

3.2 Data Access Mechanism

The system uses a structured and controlled mechanism to access and manage data. The backend server acts as the central point for handling all data operations, interacting with the database through APIs. When a user submits a query, the backend retrieves relevant information and passes it to the agent orchestration module.

Agents access required data through the orchestrator, ensuring controlled and secure data flow. The LLM interacts with the system via API calls, receiving structured prompts and returning processed responses. This approach ensures efficient data access, reduces redundancy, and maintains data integrity.

3.3 Data Retention Policies

The system follows defined data retention policies to manage stored information responsibly. User queries and generated responses may be stored for a limited period to improve system performance and enable features such as history tracking. However, sensitive data is handled carefully and is not stored permanently unless required.

Logs and interaction data may be retained temporarily for analysis and system improvement purposes. Old or unused data can be archived or deleted periodically to optimize storage and maintain system efficiency, while also ensuring compliance with data privacy considerations.

3.4 Data Migration

The system supports data migration to ensure flexibility and adaptability across different environments. Data can be transferred from local storage systems to cloud-based databases or between different database systems as required. Migration processes include data backup, transformation (if needed), and secure transfer to the target system while maintaining data integrity and consistency.

Proper validation mechanisms are applied during migration to prevent data loss or corruption. This capability ensures that the system can scale and adapt to future requirements without disrupting its functionality.

3.5 system architecture diagram

The Multi-Agent Education System uses a Python-based backend, multiple intelligent agents, and an LLM API to generate structured educational content through collaborative processing.

First, the user enters a topic or query into the system through the user interface. The Python backend receives this input and processes it before sending it to the Agent Orchestration Module, which manages the workflow between different agents.

The orchestrator assigns the task to the Researcher Agent, which retrieves relevant information from available sources or knowledge bases. This collected data is then passed to the Writer Agent, which processes and transforms the raw information into structured, clear, and meaningful educational content.

The processed content is further refined using the LLM API, which enhances reasoning, improves language quality, and ensures accurate responses. The final output is then formatted by the Post-processing Module and displayed to the user in a structured format.

This architecture ensures efficient task delegation, smooth communication between agents, and high-quality output generation using a collaborative multi-agent approach.

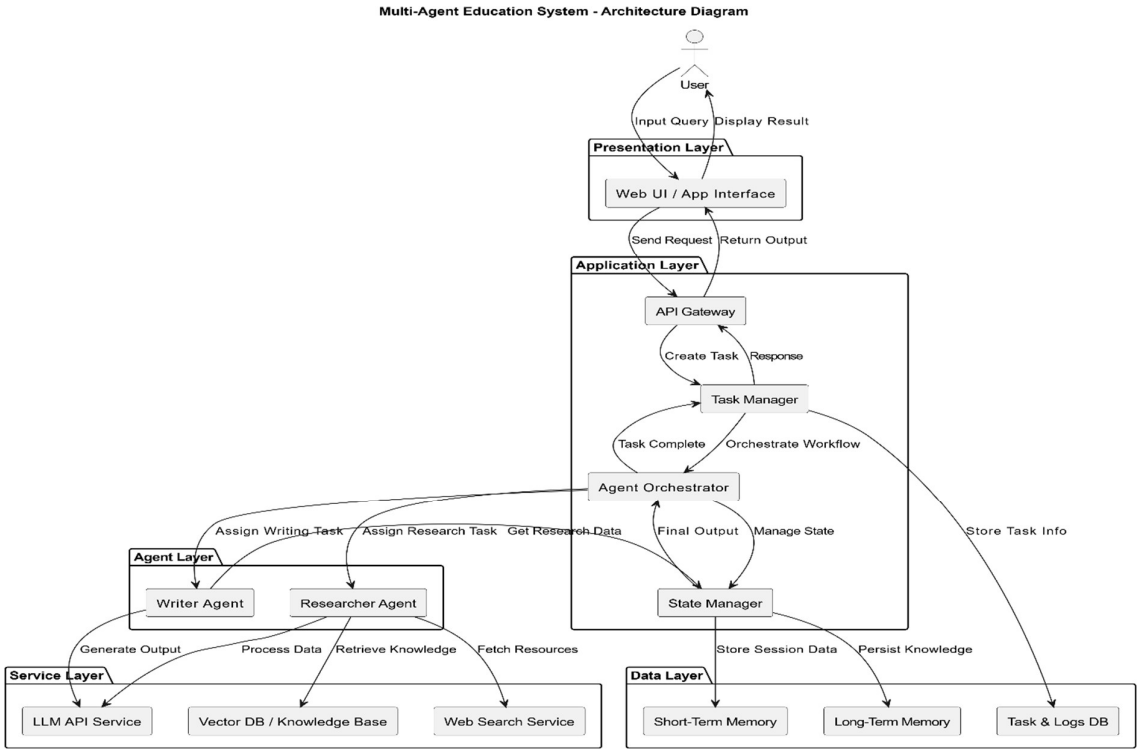
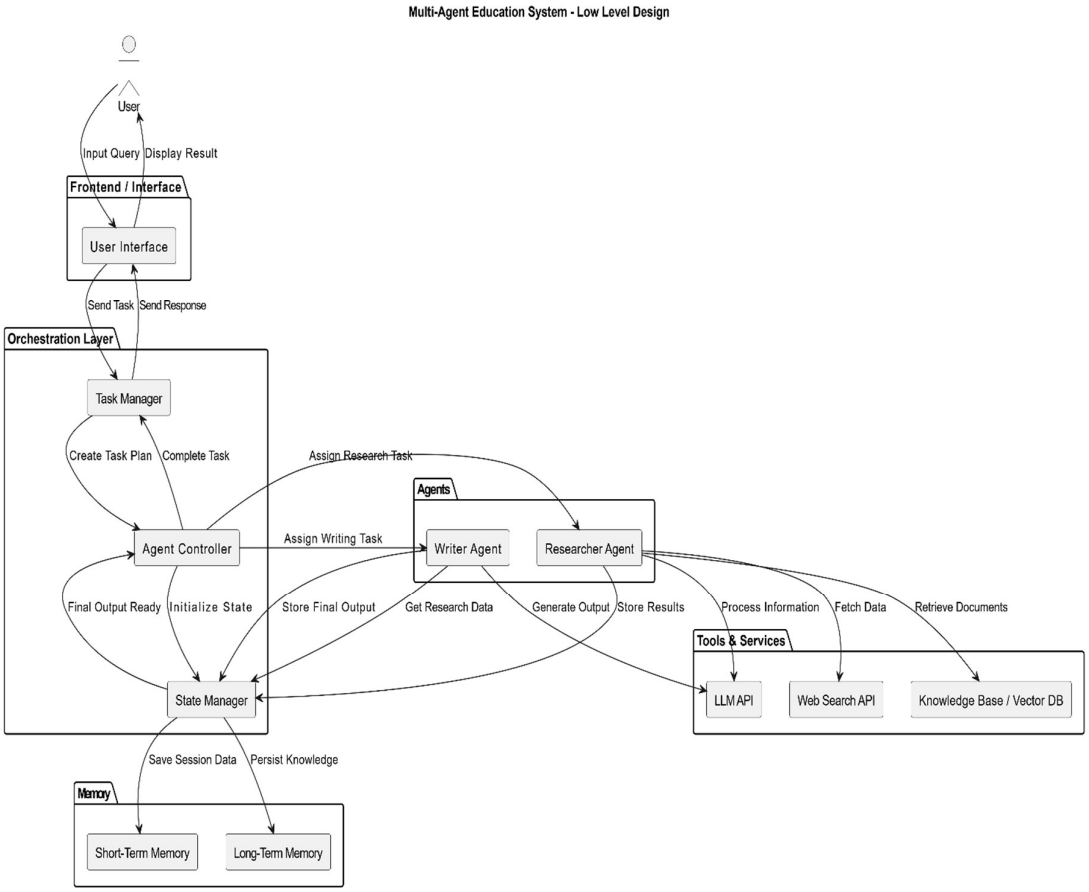


Fig 3.1: System Architechture Diagram



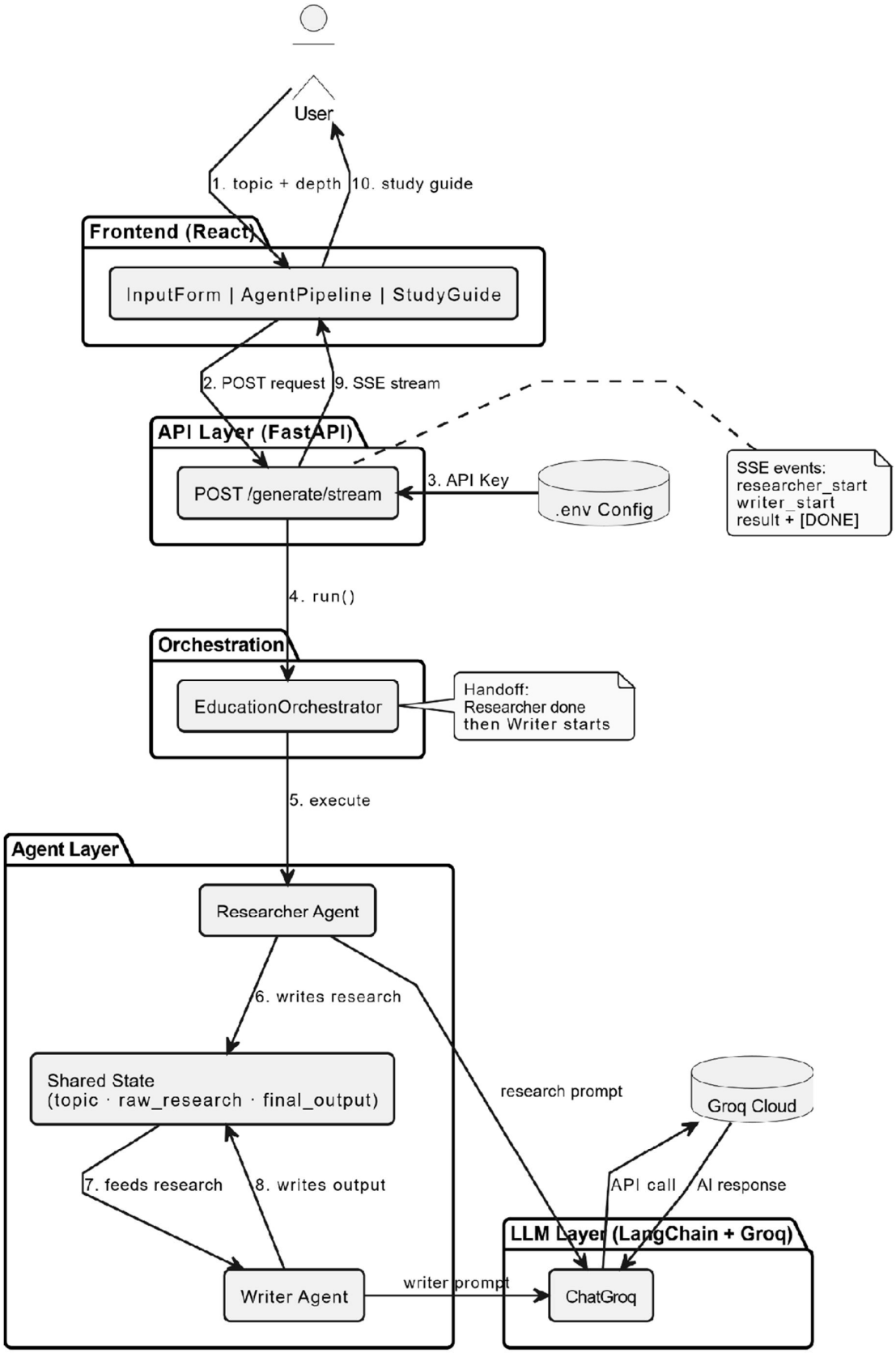


Fig 3.3: High Level Design

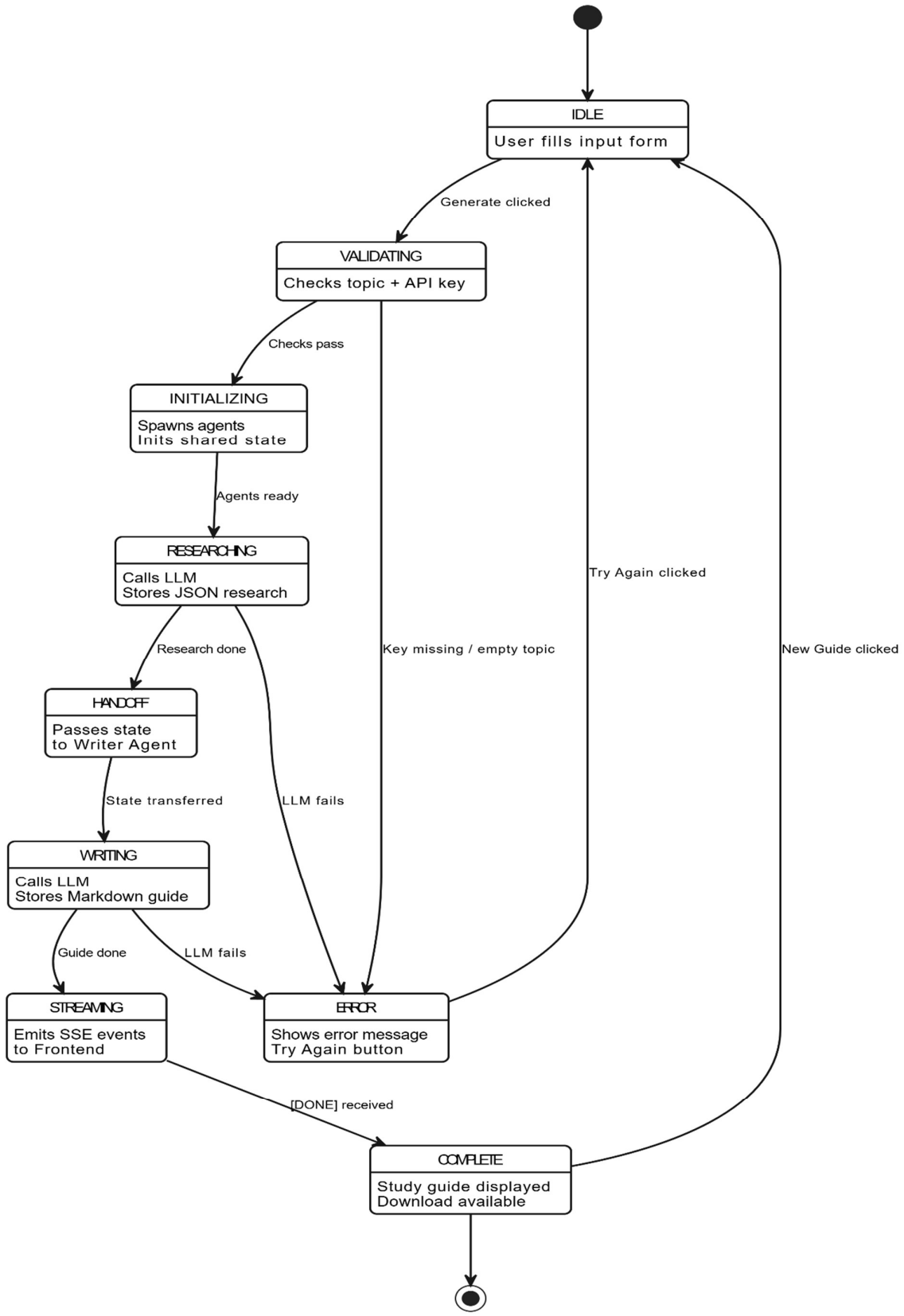


Fig 3.4: Data Flow Diagram

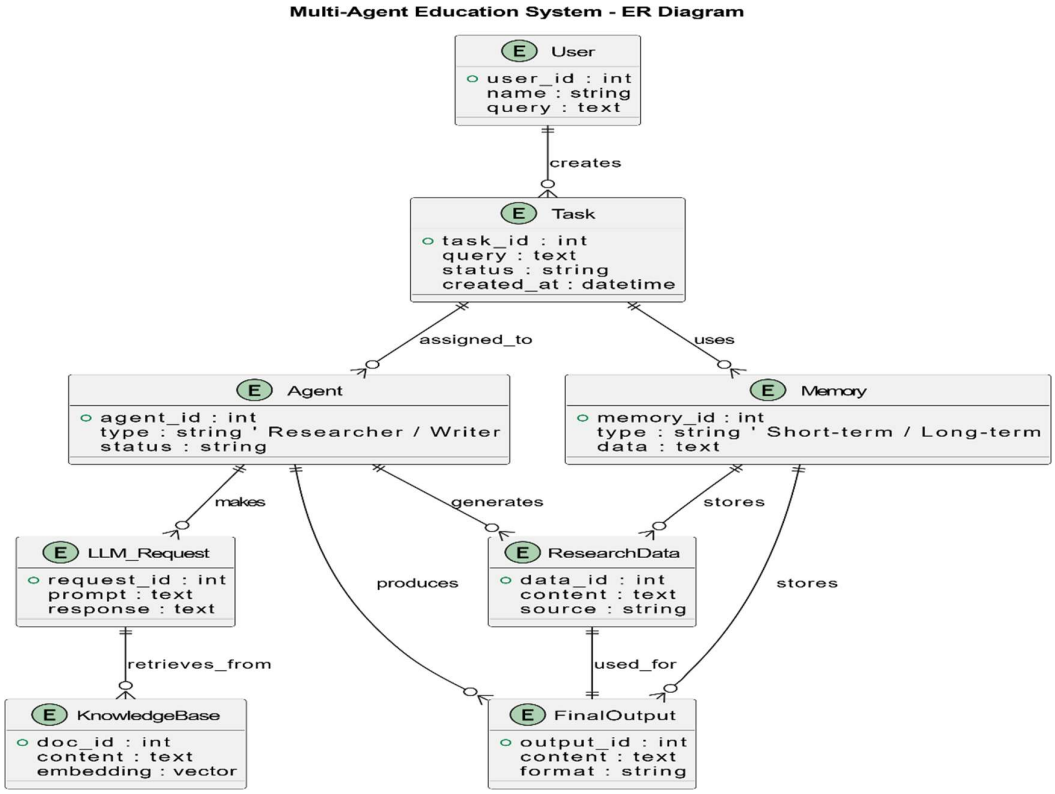


Fig 3.5: ER Diagram

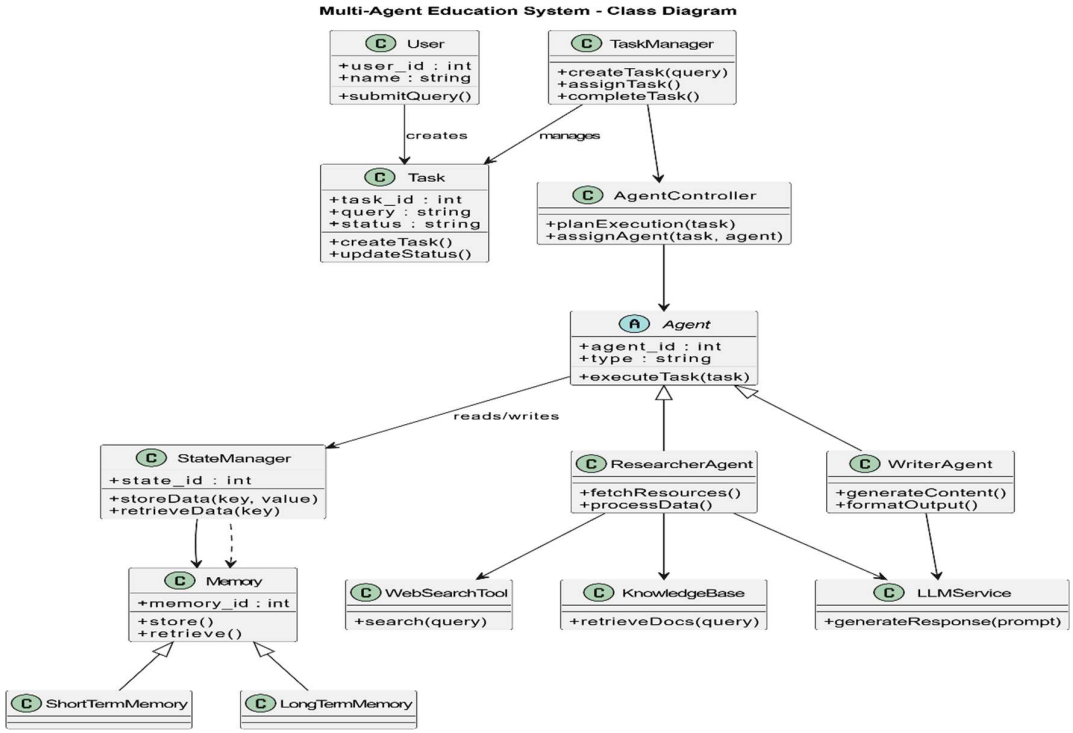


Fig 3.6: Class Diagram

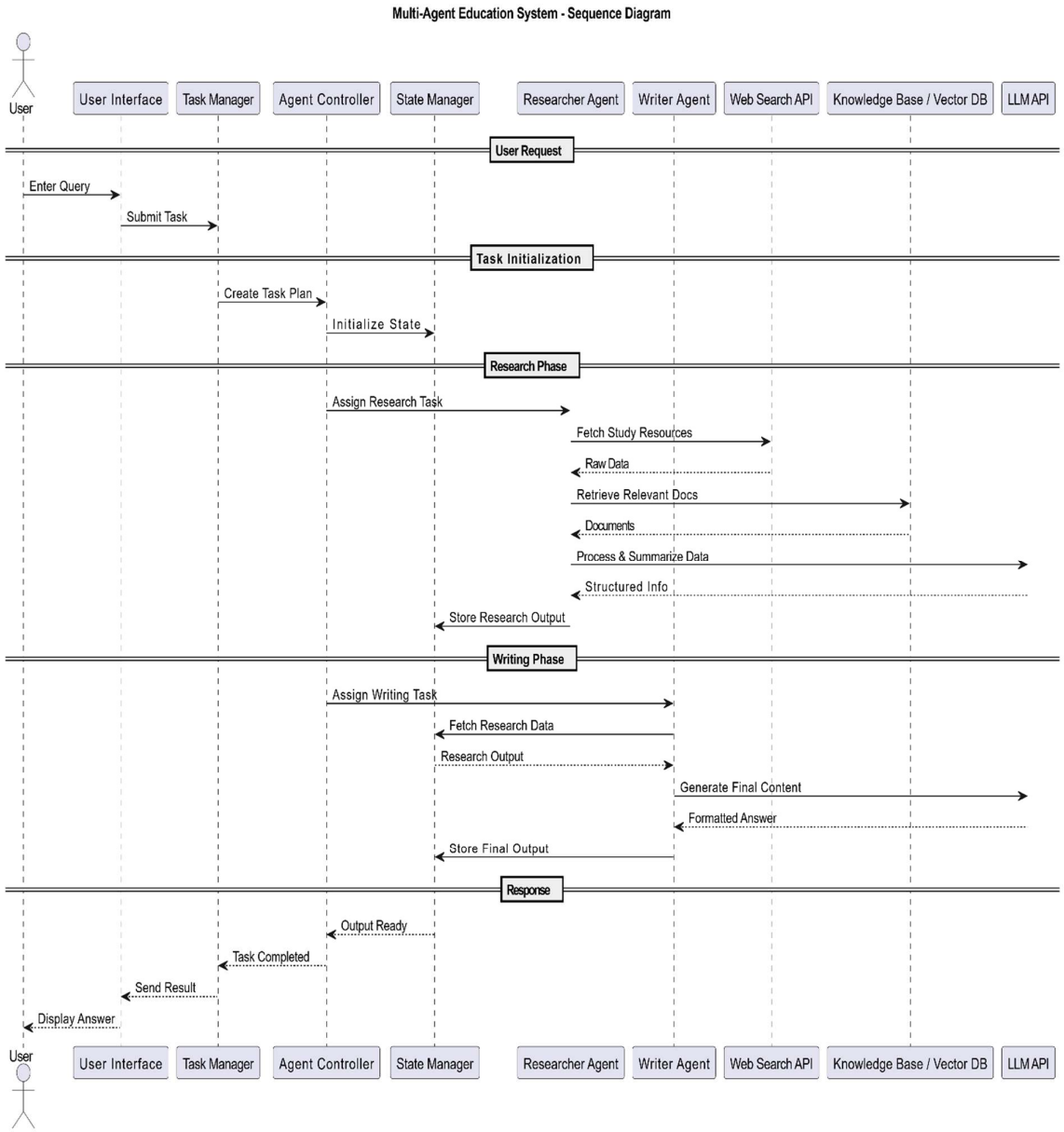


Fig 3.7: Sequence Diagram

CHAPTER 4: INTERFACES

The Multi-Agent Education System relies on well-defined interfaces to enable smooth communication between different components and external systems. These interfaces act as connection points that ensure data flows efficiently, modules interact correctly, and the overall system functions in a coordinated manner. At a high level, the system includes three primary interfaces: the User Interface, the API Interface, and the LLM Interface.

The **User Interface (UI)** serves as the primary interaction point between the user and the system. It is responsible for capturing user inputs in the form of queries or topics and displaying the generated educational responses in a structured and readable format. The UI is designed to be simple, intuitive, and user-friendly, allowing users to interact with the system without requiring technical expertise. It ensures a smooth user experience by providing clear input options and well-organized output, which enhances usability and accessibility.

The **API Interface** acts as the communication backbone of the system, enabling interaction between internal components such as the frontend, backend, agent orchestration module, and data storage systems. All requests from the user interface are sent to the backend through APIs, where they are processed and routed to the appropriate modules. Similarly, responses generated by the system are returned through APIs to the frontend for display. This interface ensures that different components remain loosely coupled, making the system more modular, scalable, and easier to maintain. It also standardizes communication, ensuring consistency in how data is transmitted and processed across the system.

The **LLM Interface** connects the system with external Large Language Model services, which play a crucial role in generating intelligent and context-aware responses. This interface is responsible for sending structured prompts, along with relevant context, to the LLM API and receiving enhanced outputs in return. It ensures secure and efficient communication with the AI model while maintaining the quality and accuracy of generated content. By integrating the LLM through a dedicated interface, the system can leverage advanced natural language processing capabilities without directly managing the complexities of the underlying model.

Together, these interfaces ensure seamless interaction between users, internal system components, and external AI services. They play a critical role in maintaining system efficiency, scalability, and reliability, ultimately enabling the system to deliver high-quality educational content in a structured and user-friendly manner.

CHAPTER 5: State and Session Management

The Multi-Agent Education System maintains state and session information to ensure continuity, consistency, and a personalized user experience during interactions. A session is created whenever a user starts interacting with the system, and it remains active until the interaction ends or times out. During this session, the system tracks user inputs, intermediate outputs, and contextual information generated by different agents. This allows the system to handle multi-step queries effectively, where the response to a current query may depend on previous interactions.

State management plays a crucial role in coordinating communication between agents. The system stores intermediate results generated by the Researcher Agent and passes them to the Writer Agent, ensuring that the context is preserved throughout the workflow. Additionally, the system may maintain short-term memory for active sessions and optional long-term memory for storing user history. Proper session handling mechanisms, such as session IDs and timeouts, are implemented to ensure efficient resource utilization and to prevent unnecessary data retention. This approach enhances user experience by providing coherent, context-aware, and continuous responses across interactions.

CHAPTER 6: NON- FUNCTIONAL REQIUREMENTS

6.1 Security Aspects

Security is a critical aspect of the Multi-Agent Education System, as it involves handling user data and interacting with external APIs. The system implements secure communication protocols to protect data transmitted between the frontend, backend, and external services. API keys used for accessing LLM services are securely stored and not exposed to unauthorized users.

User data privacy is maintained by limiting the storage of sensitive information and ensuring that only necessary data is retained. Authentication and authorization mechanisms can be implemented to control access to system features and APIs. Additionally, the system follows best practices such as input validation, protection against common vulnerabilities (e.g., injection attacks), and secure data storage techniques. These measures ensure that the system operates safely and protects both user information and system resources.

6.2 Performance Aspects

Performance is a key requirement to ensure that the system delivers fast and efficient responses to user queries. The system is designed to minimize latency by optimizing communication between components and reducing unnecessary processing steps. The use of caching significantly improves response time for frequently asked queries.

Efficient orchestration of agents ensures that tasks are executed in a streamlined manner without delays. The backend is designed to handle multiple concurrent users, making the system scalable under high load conditions. Integration with the LLM API is optimized to reduce response time while maintaining output quality. Additionally, asynchronous processing techniques can be used to improve system responsiveness.

Overall, the system is designed to provide quick, reliable, and high-quality responses, ensuring a smooth and efficient user experience even as the number of users increases.

CHAPTER 8 : References

1. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017**). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).

2. **Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020).** Language Models are Few-Shot Learners. arXiv preprint arXiv:2005.14165.

3. **Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020).** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

4. **LangChain Documentation. (2024).** LangChain Framework for LLM Applications. Available at: https://docs.langchain.com

5. **CrewAI Documentation. (2024**). Multi-Agent Orchestration Framework. Available at: https://docs.crewai.com

6. **OpenAI. (2024**). LLM API Documentation. Available at: https://platform.openai.com/docs

7. **Python Software Foundation. (2024).** Python Documentation. Available at: https://docs.python.org/3/

8. **Manning, C. D., Raghavan, P., & Schütze, H. (2008**). Introduction to Information Retrieval. Cambridge University Press.

9. **Wooldridge, M. (2009).** An Introduction to MultiAgent Systems. Wiley.

10. **Nguyen, T., Nguyen, N., & Pham, T. (2022**). AI-Based Personalized Learning Systems: A Survey. Journal of Educational Technology & Society.